Birla Institute of Technology & Science, Pilani

Work Integrated Learning Programmes Division

First Semester 2026

Assignment

(EC-1)

Course No. : SE*ZG503

Course Title : Full Stack Application Development

Weightage : 20%

Duration : 2 Weeks

No. of Pages = 9

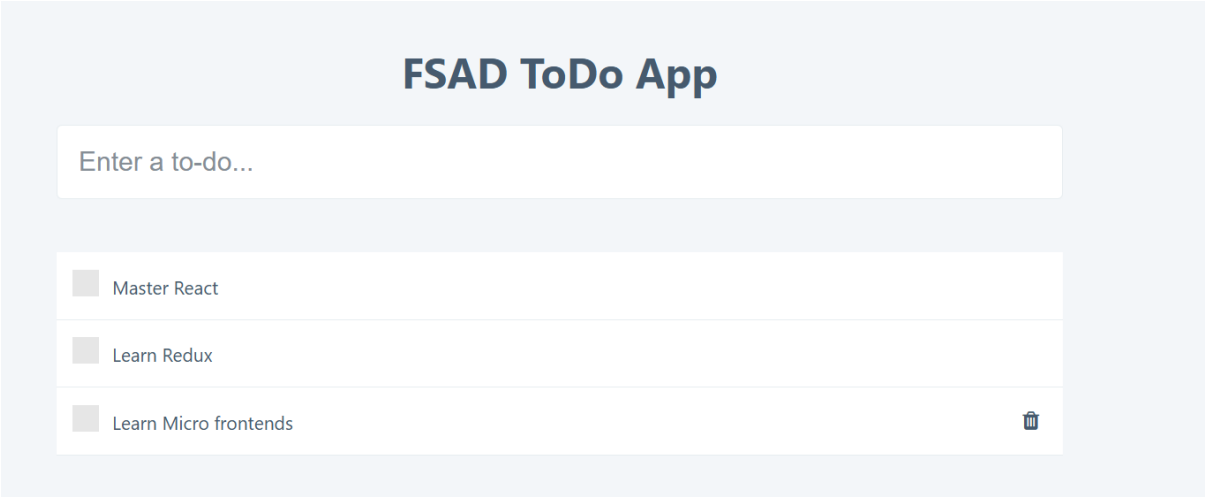
Submission Deliverable:

A ZIP file containing:

- PDF document explaining solutions to the 4 problems with necessary screen shots and code snippets.
- Code that can be compiled and run.
- Instructions to compile and run the code.

Question 1 **[4 Marks]**

1. Create a To Do application that uses local storage to save the state as JSON. It should facilitate marking the To-Do item status as completed and deleting an item.





Question 2 **[5 Marks]**

Create a React application for managing a color palette. Users of the application can add, remove, and select colors. The application should display the color palette, allowing users to remove colors and add new colors. The application structure is organized using a custom hook, **useColorPalette**.

The boilerplate code of **App.js** file and **App.cs** file are provided below. Use the boiler plate code and perform the following instructions to create the app:

Instructions

1. useColorPalette **hook:**
    I. addColor: Implement this function to add a new random color to the color palette.
    II. removeColor: Implement this function to remove a color at a specific index from the color palette.
    III. selectColor: Implement this function to set the selected color index.
    IV. useMemo: Implement this to memoize the context value and avoid unnecessary rerenders.
2. ColorPalette **function:**
    I. Use useContext to access the ColorContext and get the color palette state and related functions.
    II. Render a list of color boxes based on the colors in the palette.
    III. Apply a selected class to the currently selected color box.

IV. Add a button to remove a color.

3. ColorControls **function:**
   I. Use useContext to access the ColorContext and get the addColor function.
   II. Render a button to add a new color to the palette.
4. App **component:**
   I. Use the useColorPalette custom hook to get the color palette context value.
   II. Use the ColorContext.Provider to provide the color palette context value to the components inside.
   III. Render the ColorPalette and ColorControls components inside a parent div.

Skeleton Code:

App.css:

```
body {
  font-family: 'Arial', sans-serif;
  margin: 0;
  padding: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  height: 100vh;
  background-color: #f5f5f5;
}

.app {
  text-align: center;
}
```

```
.color-palette {
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
}

.color-box {
  position: relative;
  width: 80px;
  height: 80px;
  border-radius: 8px;
  overflow: hidden;
  cursor: pointer;
  transition: transform 0.2s ease-in-out;
}

.color-box input {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  opacity: 0;
  cursor: pointer;
}

.color-box button {
```

```
  position: absolute;
  bottom: 0;
  right: 0;
  padding: 5px;
  background-color: #fff;
  border: none;
  border-top-left-radius: 4px;
  cursor: pointer;
}

.color-box.selected {
  transform: scale(1.1);
}

.color-controls {
  margin-top: 20px;
}

.color-controls button {
  font-size: 16px;
  padding: 10px;
  cursor: pointer;
  background-color: #007bff;
  color: #fff;
  border: none;
  border-radius: 4px;
  outline: none;
}
```

Boiler Plate Code

App.js

```
import React, { useState, createContext, useContext, useMemo } from 'react';
import ReactDOM from 'react-dom';
import './App.css';

// Create a context to manage the color palette
const ColorContext = createContext();

// Custom hook: useColorPalette
const useColorPalette = () => {
 // State for managing colors and selected color
 const [colors, setColors] = useState(['#ff5733', '#33ff57', '#5733ff']);
 const [selectedColor, setSelectedColor] = useState(null);

 // Placeholder functions for adding, removing, and selecting colors
 const addColor = () => {
  // TODO: Implement the logic to add a new color to the palette
 };

 const removeColor = () => {
  // TODO: Implement the logic to remove a color from the palette
 };

 const selectColor = () => {
  // TODO: Implement the logic to select a color from the palette
```

```
  };

  // Memoize the context value to avoid unnecessary re-renders
  const contextValue = useMemo(() => ({
    colors,
    selectedColor,
    addColor,
    removeColor,
    selectColor,
  }), [colors, selectedColor]);

  return contextValue;
};

// ColorPalette component to display and customize the color palette
const ColorPalette = () => {
  // Accessing the color-related functions and data from the context
  const { colors, selectedColor, selectColor, removeColor } = useContext(ColorContext);

  return (
    <div className="color-palette">
      {/* Mapping over colors to render color boxes */}
      {colors.map((color, index) => (
        <div
          key={index}
          className={`color-box ${selectedColor === index ? 'selected' : ''}`}
          style={{ backgroundColor: color }}
          onClick={() => selectColor(index)}
```

```
      >
        <button onClick={() => removeColor(index)}>Remove</button>
      </div>
    ))}
  </div>
 );
};

// ColorControls component to add new colors to the palette
const ColorControls = () => {
 // TODO: Implement the UI for adding new colors to the palette
};

// App component using the ColorProvider and rendering ColorPalette and
ColorControls
const App = () => {
 // Use the custom hook

 return (
  <p>
   Coding playground of the given task
  </p>
 );
};

// Helper function to generate a random hex color
const getRandomColor = () => {
 const letters = '0123456789ABCDEF';
```

```
  let color = '#';
  for (let i = 0; i < 6; i++) {
    color += letters[Math.floor(Math.random() * 16)];
  }
  return color;
};

export default App;
```

Question 3 **[7 Marks]**

Implement a Microfrontend shell using Webpack module federation. Create two micro apps **FSAD** and **ApiProducts**. Display FSAD Syllabus in the FSAD microapp and ApiProducts Syllabus in the ApiProducts micro app.

Question 4 **[4 Marks]**

You must create a **WebAssembly module** that exposes two functions:

1. add(a, b) → returns a + b
2. multiply(a, b) → returns a * b

Then write a **JavaScript host program** that:

- Loads the .wasm file
- Calls both functions
- Prints the results

You may implement the WebAssembly module in any programming language **C, C#, Java, Rust, or Go**. The host program must be in **JavaScript** (Node.js or browser).